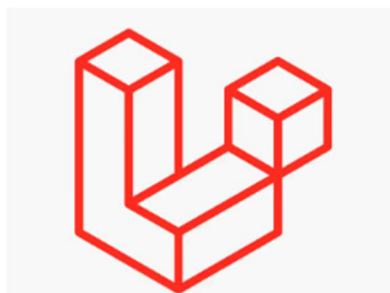


ระยะยาว พัฒนาโดย Taylor Otwell ภายใต้ไลเซนส์แบบ MIT และเปิดเผยแพร่โค้ดบน GitHub จึงมีชุมชนนักพัฒนาขนาดใหญ่ แพ็กเกจเสริมหลากหลาย และมีการอัปเดตอย่างต่อเนื่อง

แนวคิดหลักของ Laravel คือการแยกความรับผิดชอบของโค้ดออกเป็น 3 ส่วนชัดเจน ได้แก่

- Model จัดการข้อมูลและตรรกะที่เกี่ยวข้องกับฐานข้อมูล
- View แสดงผลส่วนติดต่อผู้ใช้
- Controller ประสานงานคำขอจากผู้ใช้กับตรรกะของระบบและแบบจำลองข้อมูล

ด้วยการแยกชั้นเช่นนี้ โค้ดอ่านง่าย ทดสอบได้สะดวก และทำงานร่วมกันเป็นทีมได้มีประสิทธิภาพ นอกจากนี้ Laravel ยังมาพร้อมเครื่องมือที่จำเป็นครบวงจร เช่น Routing, Middleware, Validation, Eloquent ORM, Migrations, Authentication/Authorization (Policy & Gate), Queue/Jobs, Scheduler, Events & Broadcasting, Notifications, Testing, Localization รวมถึงคำสั่ง Artisan CLI ที่ช่วยเร่งงานพัฒนาและบำรุงรักษา



รูปที่ 2.7 ไอคอนของ Framework Laravel

### 2.2.5 Inertia.js [8]

Inertia.js คือไลบรารีที่ทำหน้าที่เป็น “สะพาน” ระหว่างเฟรมเวิร์กฝั่งเซิร์ฟเวอร์ (เช่น Laravel) กับเฟรมเวิร์ก UI ฝั่งไคลเอนต์ (เช่น React) เพื่อให้ได้ประสบการณ์แบบ Single-Page Application (SPA) โดย ไม่ต้องสร้าง REST/GraphQL API แยก และยังคงใช้เส้นทาง (routing), มิดเดิลแวร์, และการยืนยันตัวตนของ Laravel ได้ตามปกติ

คุณสมบัติเด่น

- SPA โดยไม่ต้องทำ API: ลดงานประกอบระบบ แยกน้อย บำรุงรักษาง่าย
- Form & Validation: ส่งฟอร์ม/รับ error จาก Laravel ได้ตรง ๆ (error bag, flash)
- Partial Reload: รีเฟรชเฉพาะข้อมูลที่เปลี่ยน ลดโหลดเครือข่าย

- Shared Props: แשרข้อมูลกลาง (ผู้ใช้ปัจจุบัน, สิทธิ์, locale, แง้งเต็อน) ใ้ทุกหน้า
- Navigation/History: จัดการเปลี่ยนหน้าและ progress bar ลื่นไหลเหมือน SPA
- SEO/Internal: ยังใช้เส้นทางของ Laravel ได้ ชัดเจนและเป็นระเบียบ



รูปที่ 2.8 ไอคอนของ Inertia.js

### 2.2.6 React [9]

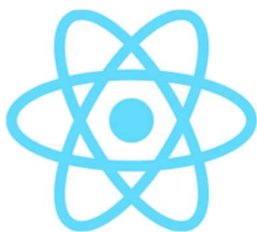
React คือไลบรารี JavaScript ที่พัฒนาโดย Facebook (ปัจจุบัน Meta) สำหรับสร้างส่วนติดต่อผู้ใช้ที่มีประสิทธิภาพและปรับตัวได้สูง โดยยึดแนวคิด Component-Based Architecture แยก UI ออกเป็นชิ้นส่วนย่อยเพื่อความ นำกลับมาใช้ซ้ำ (Reusable) และ ดูแลง่าย (Maintainable) แต่ละคอมโพเนนต์สามารถจัดการเหตุการณ์ (Event Handling) ติดต่อกับ API และควบคุมข้อมูลของตนเองผ่าน State และรับค่าจากภายนอกผ่าน Props

React ใช้กลไก Virtual DOM เพื่อเปรียบเทียบความเปลี่ยนแปลงแล้วอัปเดตเฉพาะส่วนที่จำเป็น ทำให้เรนเดอร์รวดเร็วและมีประสิทธิภาพ ถูกเริ่มต้นโดย Jordan Walke ภายในทีม Facebook และเปิดเป็น โอเพนซอร์ส ตั้งแต่ปี 2013 จนกลายเป็นมาตรฐานสำคัญของงานพัฒนา UI สมัยใหม่

คุณลักษณะเด่น

- Component-Based Architecture: แยกส่วนหน้าจออกเป็นคอมโพเนนต์ย่อย จัดระเบียบโค้ด และทดสอบได้ง่าย
- Reusable & Maintainable: สร้างคอมโพเนนต์ครั้งเดียว ใช้ซ้ำได้หลายหน้า ลดโค้ดซ้ำ
- Virtual DOM: อัปเดตเฉพาะส่วนที่เปลี่ยน เพิ่มความลื่นไหล
- Unidirectional Data Flow: ข้อมูลไหลทิศเดียวแบบ *Parent* → *Child* ผ่าน Props ทำให้ตามรอยและดีบั้งง่าย
- JSX: ภาษาลักษณะคล้าย HTML ผสานกับ JavaScript เพื่ออธิบาย UI แบบเชิงประกาศ (Declarative)

- Hooks (เช่น useState, useEffect) ช่วยใช้ตรรกะมีสถานะแบบฟังก์ชันโดยไม่ต้องเขียนคลาส



รูปที่ 2.9 ไอคอนของ React

## 2.3 ฐานข้อมูล [10]

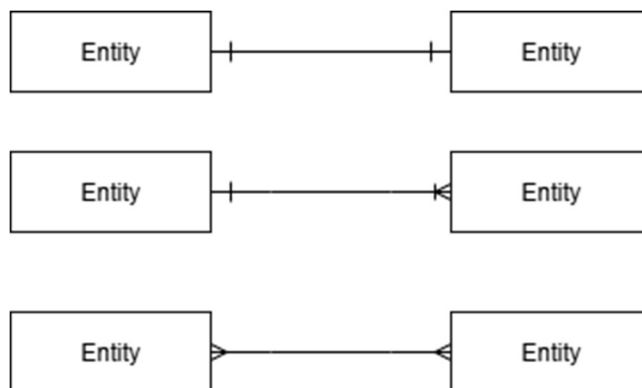
ระบบจัดการฐานข้อมูล (Database Management System: **DBMS**) คือซอฟต์แวร์สำหรับจัดเก็บ จัดระเบียบ และบริหารข้อมูลอย่างเป็นระบบ โดยกำหนดโครงสร้างและความสัมพันธ์ของข้อมูลที่เก็บในรูปแบบดิจิทัลบนคอมพิวเตอร์ DBMS ช่วยควบคุมการเพิ่ม-แก้ไข-ลบ-ค้นคืนข้อมูลให้ถูกต้องสอดคล้องและมีประสิทธิภาพ รองรับการจัดการข้อมูลปริมาณมาก การเข้าถึงที่รวดเร็ว และการสร้างรายงานได้อย่างเป็นระเบียบ พร้อมทั้งมีมาตรการรักษาความปลอดภัย เช่น การกำหนดสิทธิ์ผู้ใช้ การตรวจสอบการใช้งาน และการเข้ารหัส ทำให้ข้อมูลมีความถูกต้อง เชื่อถือได้ และปลอดภัย

### 2.3.1 คีย์ของฐานข้อมูล

1) คีย์ (Key) คือกลไกสำคัญที่ใช้ระบุตัวตนของระเบียนและเชื่อมโยงความสัมพันธ์ระหว่างตาราง เพื่อให้การอ้างอิง ค้นหา แก้ไข และประมวลผลข้อมูลทำได้ถูกต้องและมีประสิทธิภาพ การกำหนดคีย์ที่เหมาะสมช่วยลดความซ้ำซ้อนและรักษาความสอดคล้องของข้อมูล (referential integrity) ภายในฐานข้อมูล

2) คีย์หลัก (Primary Key: PK) เป็นคอลัมน์หรือชุดคอลัมน์ที่ระบุระเบียนได้ “ไม่ซ้ำ” และ ห้ามเป็นค่าว่าง (NULL) ในตารางเดียวกันหนึ่งระเบียนต้องมีค่า PK เฉพาะของตน ใช้เป็นจุดอ้างอิงให้ตารางอื่น ๆ และมักสร้างดัชนีโดยอัตโนมัติเพื่อให้ค้นหาได้รวดเร็ว

คีย์นอก (Foreign Key: FK) เป็นคอลัมน์ที่เก็บค่าของ PK/Unique จากตารางอื่น เพื่อเชื่อมความสัมพันธ์ระหว่างตารางและบังคับความสอดคล้องของข้อมูล สามารถกำหนดพฤติกรรมเมื่อมีการแก้ไข/ลบข้อมูลต้นทางได้ เช่น ON UPDATE/DELETE CASCADE, SET NULL, หรือ RESTRICT



รูปที่ 2.10 ตัวอย่างความสัมพันธ์ของตารางในฐานข้อมูล

### 2.3.2 ระบบจัดการฐานข้อมูล (MySQL)

MySQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System: RDBMS) ที่จัดเก็บข้อมูลในรูปแบบ ตาราง (Table) โดยแต่ละตารางประกอบด้วย แถว/ระเบียน (Row/Record) และ คอลัมน์/ฟิลด์ (Column/Field) การเชื่อมโยงข้อมูลระหว่างตารางทำผ่าน คีย์ เช่น *Primary Key* และ *Foreign Key* แทนการเก็บข้อมูลซ้ำซ้อน ทำให้โครงสร้างข้อมูลเป็นระเบียบและคงความถูกต้อง (data integrity)

ตัวระบบมีเครื่องมือสำหรับ เพิ่ม-แก้ไข-ลบ-ค้น ข้อมูล จัดทำข้อจำกัด (constraints) และดัชนี (indexes) เพื่อให้สืบค้นได้รวดเร็ว รองรับการจัดกลุ่มและความสัมพันธ์ของข้อมูลตามความต้องการของงาน ส่งผลให้การบริหารข้อมูลมีประสิทธิภาพ ใช้งานง่าย ขยายระบบได้ และเข้าถึงข้อมูลได้รวดเร็ว เหมาะกับงานที่ต้องการความน่าเชื่อถือและการจัดการข้อมูลจำนวนมากอย่างเป็นระบบ

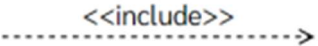
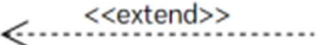
## 2.4 ยูเอ็มแอล (Unified Modeling Language: UML)

ภาษา UML (Unified Modeling Language) เป็นภาษาที่ใช้อธิบายแบบจำลองเชิงวัตถุ โดยเป็นภาษามาตรฐานสำหรับสร้างแบบพิมพ์เขียวให้แก่ระบบงาน UML สามารถใช้ในการสร้างมุมมอง กำหนดรายละเอียด สร้างระบบงาน และจัดทำเอกสารอ้างอิงของระบบงาน ทำให้ผู้วิเคราะห์ระบบและผู้ที่เกี่ยวข้องระบบตรงกัน

### 2.4.1 แผนภาพ Use Case (Use Case diagram)

เป็นแผนภาพที่ใช้แสดงปฏิสัมพันธ์ระหว่างระบบงานและสิ่งที่ยู่นอกระบบงาน ใช้ แสดงภาพรวมและความสามารถโดยรวมของระบบ โดยไม่สนใจกลไกการทำงานหรือเทคนิคการทำงาน เป็นแผนภาพเริ่มต้นในการวิเคราะห์ระบบ สัญลักษณ์ที่ใช้ในแผนภาพ Use Case เป็นดังตารางที่ 2.1

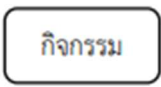
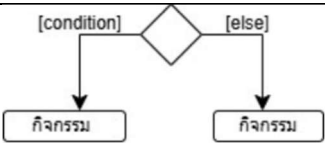
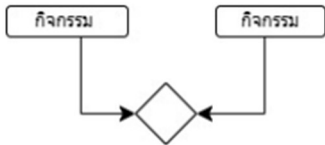
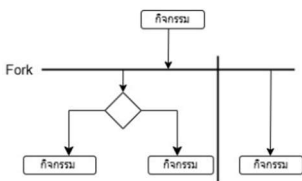
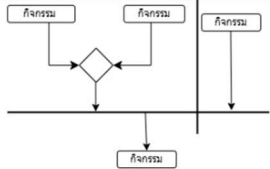
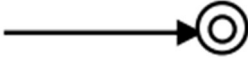
ตารางที่ 2.1 สัญลักษณ์ที่ใช้ในแผนภาพ Use Case

| สัญลักษณ์                                                                           | ชื่อ        | ความหมาย                                                                      |
|-------------------------------------------------------------------------------------|-------------|-------------------------------------------------------------------------------|
|    | Use case    | ความสามารถของระบบ                                                             |
|    | Actor       | ผู้ที่เกี่ยวข้องกับระบบ อาจเป็นผู้ใช้งานหรือระบบอื่น                          |
|    | Association | การติดต่อระหว่าง Actor กับ Use Case                                           |
|  | Include     | ความสัมพันธ์ที่ Use Case หนึ่งต้องพึ่งพาความสามารถของ Use Case อื่น           |
|  | Extend      | ความสัมพันธ์ที่ Use Case หนึ่งอาจถูก กระตุ้นด้วยบางเหตุการณ์จาก Use Case อื่น |

#### 2.4.2 แผนภาพ Activity (Activity diagram)

เป็นแผนภาพที่แสดงกระบวนการทำงานของระบบ เพื่อการวิเคราะห์และออกแบบ ระบุซอฟต์แวร์ การสื่อสารกับผู้ที่มีส่วนเกี่ยวข้องกับการทำงานของระบบ และการทดสอบระบบ ซอฟต์แวร์ สัญลักษณ์ที่ใช้ในแผนภาพ Activity เป็นดังตารางที่ 2.2

ตารางที่ 2.2 สัญลักษณ์ที่ใช้ในแผนภาพ Activity

| สัญลักษณ์                                                                           | ชื่อ          | ความหมาย                                                                       |
|-------------------------------------------------------------------------------------|---------------|--------------------------------------------------------------------------------|
|    | Initial State | จุดเริ่มต้น                                                                    |
|    | Activity      | กิจกรรม                                                                        |
|    | Control Flow  | การไหลของกิจกรรม                                                               |
|  | Branch        | กรณีใช้ในการแสดงเงื่อนไขที่มีทางเลือกแตกต่างกัน                                |
|  | Merge         | กรณีใช้เมื่อเส้นทางที่แยกออกไปกลับมารวมกันอีกครั้ง                             |
|  | Fork          | กรณีใช้เมื่อต้องการแสดงกิจกรรมที่สามารถทำงานพร้อมกัน                           |
|  | Join          | กรณีใช้เมื่อกิจกรรมที่ทำพร้อมกันเสร็จสิ้นและกลับมารวมกันก่อนดำเนินกิจกรรมต่อไป |
|  | Final State   | จุดสิ้นสุด                                                                     |